

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Parsing

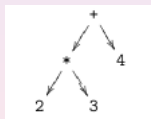
Monadic parser combinators

The parser type

What is a parser?

A program which takes a string and outputs an entity (e.g., a tree) which makes the string's syntactic structure explicit.

Example (A tree corresponding to the string "2*3+4")



The parser type

Example

```
type Parser = String -> Tree
```

Remark

We could regard a parser as a function from strings to trees, which maps flat input to a structured output.

The parser type

Example

```
type Parser = String -> (Tree, String)
```

Remark

The parser does not always consume the whole input string. If a part of the input string is not parsed, it will be returned as the second component of the result tuple.

The parser type

Example

```
type Parser = String -> [(Tree, String)]
```

Remark

The result of the parsing process is not always unique (e.g., in case of natural languages), so we'll return all possible parses. A singleton list returned means success, an empty list returned means failure.

The parser type

Example

```
type Parser a = String -> [(a, String)]
```

Remark

To preserve generality, we should abstract over type and allow the parser to return something different from a tree as well.

The parser type

Example

```
type Parser a = String -> [(a, String)]
```

A poem by Fritz Ruehs

*A parser of things
Is a function from strings
To lists of pairs
Of things and strings.*

The parser type

Example (In order to keep consistency)

```
import Control.Applicative
import Data.Char
```

```
newtype Parser a = P (String -> [(a, String)])
```

Remark

P is just a dummy constructor.

Building blocks for parsers

`parse`: applying a parser

```
parse :: Parser a -> String -> [(a, String)]  
parse (P p) inp = p inp
```

Remark

Function `parse` applies a parser to an input string. To do this, it removes the dummy constructor `P`.

Building blocks for parsers

item: get first item

```
item :: Parser Char
item = P (\inp -> case inp of
                []      -> []
                (x:xs) -> [(x,xs)])
```

Remark

Returned result: first character. Fails on empty string.

Example (Call item)

```
> parse item ""
[]
```

```
> parse item "xyz"
[('x', "yz")]
```

How to combine (=sequence) parsers?

Making Parser a Functor

```
instance Functor Parser where
  fmap g p = P (\inp -> case parse p inp of
                        []          -> []
                        [(v,out)]   -> [(g v, out)])
```

Remark

`fmap` applies a function to the result value if the parser succeeds; otherwise it propagates failure.

Example

```
> parse (fmap toUpper item) "abc"
[('A', "bc")]
```

```
> parse (fmap toUpper item) ""
[]
```

How to combine (=sequence) parsers?

Making Parser an Applicative

```
instance Applicative Parser where  
  pure v = P (\inp -> [(v, inp)])
```

Remark

`pure` transforms a value into a parser that always succeeds with this value as its result, but doesn't consume anything from the input string.

Example

```
> parse (pure 1) "abcdef"  
[(1, "abcdef")]
```

How to combine (=sequence) parsers?

Making Parser an Applicative

```
instance Applicative Parser where
  pg <*> px = P (\inp -> case parse pg inp of
    []          -> []
    [(g, out)] -> parse (fmap g px) out)
```

Remark

`<*>` applies a parser that returns a function to a parser that returns an argument to get a parser returning the result of applying the function to the argument. It only succeeds if all the components succeed.

How to combine (=sequence) parsers?

Making Parser an Applicative

```
instance Applicative Parser where
  pg <*> px = P (\inp -> case parse pg inp of
    []          -> []
    [(g, out)] -> parse (fmap g px) out)
```

Example

```
--consumes 3 characters, drops the second
three :: Parser (Char, Char)
three = pure g <*> item <*> item <*> item
      where g x y z = (x,z)
```

```
> parse three "abcdef"
[('a', 'c'), "def"]
```

How to combine (=sequence) parsers?

Making Parser an Applicative

```
instance Applicative Parser where
  pg <*> px = P (\inp -> case parse pg inp of
    []          -> []
    [(g, out)] -> parse (fmap g px) out)
```

Example

```
--consumes 3 characters, drops the second
three :: Parser (Char, Char)
three = pure g <*> item <*> item <*> item
      where g x y z = (x,z)
```

```
> parse three "ab"
[]
```

How to combine (=sequence) parsers?

Making Parser a Monad

```
instance Monad Parser where
  --(>>=) :: Parser a -> (a -> Parser b) -> Parser b
  p >>= f = P (\inp -> case parse p inp of
                        []          -> []
                        [(v,out)] -> parse (f v) out)
```

Remark

Parser $p \gg= f$ fails if the application of the parser p to the input string inp fails; otherwise, it applies the function f to the result value v to give another parser $f\ v$, which is then applied to the output string out (which is produced by the first parser) to give the final result.

How to combine (=sequence) parsers?

Using `do` to sequence parsers

```
three :: Parser (Char,Char)
three = do x <- item
           item
           z <- item
           return (x,z)
```

Making choices

Combine parser to express alternatives

Apply one parser to the input string; if this fails, then apply another one to the same input instead

The general `Alternative`

```
class Applicative f => Alternative f where
  empty :: f a
  (<|>) :: f a -> f a -> f a
```

Remark

`empty` can be seen as "alternative that has failed".

`<|>` is a choice operator

How to combine (=sequence) parsers?

Making Parser an Alternative

```
instance Alternative Parser where
  -- empty :: Parser a
  empty = P (\inp -> [])

  -- (<|>) :: Parser a -> Parser a -> Parser a
  p <|> q = P (\inp -> case parse p inp of
                        []          -> parse q inp
                        [(v,out)] -> [(v,out)])
```

Example

```
> parse empty "abc"
[]
```

How to combine (=sequence) parsers?

Making Parser an Alternative

```
instance Alternative Parser where
  -- empty :: Parser a
  empty = P (\inp -> [])

  -- (<|>) :: Parser a -> Parser a -> Parser a
  p <|> q = P (\inp -> case parse p inp of
                        []          -> parse q inp
                        [(v,out)] -> [(v,out)])
```

Example

```
> parse (item <|> return 'd') "abc"
[('a', "bc")]
```

How to combine (=sequence) parsers?

Making Parser an Alternative

```
instance Alternative Parser where
  -- empty :: Parser a
  empty = P (\inp -> [])

  -- (<|>) :: Parser a -> Parser a -> Parser a
  p <|> q = P (\inp -> case parse p inp of
                        []          -> parse q inp
                        [(v,out)] -> [(v,out)])
```

Example

```
> parse (empty <|> return 'd') "abc"
[('d', "abc")]
```

Derived (=assembled) primitives

Satisfying a predicate

```
sat :: (Char -> Bool) -> Parser Char
sat p = do x <- item
        if p x then return x else empty
```

Example (Recognizing single digits)

```
digit :: Parser Char
digit = sat isDigit -- from Data.Char
```

Derived (=assembled) primitives

Satisfying a predicate

```
sat :: (Char -> Bool) -> Parser Char
sat p = do x <- item
        if p x then return x else empty
```

Example (Recognizing letters)

```
lower :: Parser Char
lower = sat isLower
```

```
upper :: Parser Char
upper = sat isUpper
```

Derived (=assembled) primitives

Satisfying a predicate

```
sat :: (Char -> Bool) -> Parser Char
sat p = do x <- item
        if p x then return x else empty
```

Example (Recognizing letters)

```
letter :: Parser Char
letter = sat isAlpha

alphanum :: Parser Char
alphanum = sat isAlphaNum
```

Derived (=assembled) primitives

Satisfying a predicate

```
sat :: (Char -> Bool) -> Parser Char
sat p = do x <- item
        if p x then return x else empty
```

Example (Recognizing characters)

```
char :: Char -> Parser Char
char x = sat (== x)
```

Example

```
> parse (char 'a') "abc"
[('a', "bc")]
```

Derived (=assembled) primitives

Satisfying a predicate

```
string :: String -> Parser String
string [] = return []
string (x:xs) = do char x
                   string xs
                   return (x:xs)
```

Example

```
> parse (string "abc") "abcdef"
[("abc", "def")]
> parse (string "abc") "ab1234"
[]
```

Derived (=assembled) primitives

Applying a parser p more times

```
class Applicative f => Alternative f where
  many :: f a -> f [a]
  some :: f a -> f [a]
```

```
some x = pure (:) <*> x <*> many x -- min. once
many x = some x <|> pure [] -- 0 or more times
```

Example

```
> parse (many digit) "123abc"
[("123", "abc")]
> parse (many digit) "abc"
[("", "abc")]
> parse (some digit) "abc"
[]
```

Derived (=assembled) primitives

Recognizing identifiers

```
ident :: Parser String
ident = do x <- lower
          xs <- many alphanum
          return (x:xs)
```

Example

```
> parse ident "abc def"
[("abc", "def")]
```

Derived (=assembled) primitives

Recognizing natural numbers

```
nat :: Parser Int
nat = do xs <- some digit
      return (read xs)
```

Example

```
> parse nat "123 abc"
[(123, "abc")]
```

Derived (=assembled) primitives

Recognizing spaces

```
space :: Parser ()  
space = do many (sat isSpace)  
         return ()
```

Example

```
> parse space "   abc"  
[(), "abc"]
```

Derived (=assembled) primitives

Recognizing integers

```
int :: Parser Int
int = do char '-'
        n <- nat
        return (-n)
<|> nat
```

Example

```
> parse int "-123 abc"
[(-123, "abc")]
```

Handling spaces

Ignore spaces before and after applying a parser

```
token :: Parser a -> Parser a
token p = do space
            v <- p
            space
            return v
```

```
natural :: Parser Int
natural = token nat
```

```
symbol :: String -> Parser String
symbol xs = token (string xs)
```

Arithmetic expressions

Definition (Grammar)

Set of rules describing how correct strings are constructed

A grammar for the language of arithmetic expressions

```
expr ::= expr + expr | term  
term ::= term * term | factor  
factor ::= ( expr ) | nat  
nat ::= 0 | 1 | 2 | ...
```

Arithmetic expressions

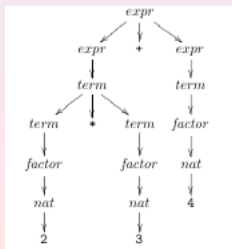
Definition (Parse tree)

A tree representing the parsed structure of string

A grammar for the language of arithmetic expressions

```
expr ::= expr + expr | term
term ::= term * term | factor
factor ::= ( expr ) | nat
nat ::= 0 | 1 | 2 | ...
```

Parse tree for $2 * 3 + 4$



Arithmetic expressions

A grammar for the language of arithmetic expressions

```
expr ::= term ( + expr |  $\epsilon$  )  
term ::= factor ( * term |  $\epsilon$  )  
factor ::= ( expr ) | nat  
nat ::= 0 | 1 | 2 | ...
```

Recognizing integers

```
expr :: Parser Int  
expr = do t <- term  
         do symbol "+"  
            e <- expr  
            return (t + e)  
<|> return t
```

Arithmetic expressions

A grammar for the language of arithmetic expressions

```
expr ::= term ( + expr |  $\epsilon$  )  
term ::= factor ( * term |  $\epsilon$  )  
factor ::= ( expr ) | nat  
nat ::= 0 | 1 | 2 | ...
```

Recognizing integers

```
term :: Parser Int  
term = do f <- factor  
        do symbol "*"   
            t <- term  
            return (f * t)  
<|> return f
```

Arithmetic expressions

A grammar for the language of arithmetic expressions

```
expr ::= term ( + expr |  $\epsilon$  )  
term  ::= factor ( * term |  $\epsilon$  )  
factor ::= ( expr ) | nat  
nat    ::= 0 | 1 | 2 | ...
```

Recognizing integers

```
factor :: Parser Int  
factor = do symbol "("  
            e <- expr  
            symbol ")"  
            return e  
<|> natural
```

Arithmetic expressions

Evaluating expressions

```
eval :: String -> Int
eval xs = case (parse expr xs) of
    [(n, [])]    -> n
    [(_, out)]   -> error ("Unused " ++ out)
    []           -> error "Invalid input"
```

Example

```
> eval "2*3+4"
10
```

Arithmetic expressions

Evaluating expressions

```
eval :: String -> Int
eval xs = case (parse expr xs) of
    [(n, [])]    -> n
    [(_, out)]   -> error ("Unused " ++ out)
    []           -> error "Invalid input"
```

Example

```
> eval "2*(3+4) "
14
```

Arithmetic expressions

Evaluating expressions

```
eval :: String -> Int
eval xs = case (parse expr xs) of
    [(n, [])]    -> n
    [(_, out)]   -> error ("Unused " ++ out)
    []           -> error "Invalid input"
```

Example

```
> eval "2*3^4"
*** Exception: Unused ^4
```

Arithmetic expressions

Evaluating expressions

```
eval :: String -> Int
eval xs = case (parse expr xs) of
    [(n, [])]    -> n
    [(_, out)]   -> error ("Unused " ++ out)
    []           -> error "Invalid input"
```

Example

```
> eval "one plus two"
*** Exception: Invalid input
```

Summary

- ▶ A parser is a program which takes flat strings as input and builds entities giving the structure of those strings
- ▶ A parser could be seen as a function from strings to structures
- ▶ monads help us build complex parsers by combining simple ones and gracefully dealing with exceptions